Extraction Optimization Study

Recipe and invoice extraction: hold 98% accuracy first, then cut time and cost. A follow-up to the Milestone 3 audit, measured live on 27 documents

Prepared for	Clayton Linville (Product Owner) and the development team (Viktor Musiienko, PM). Section 9 is the hand-off for the developers.
Date	October 13, 2026 (runs on October 12 and 13). Section 11, a test on 58 new real documents, added October 15.
Builds on	Milestone 3 Audit Report, section 5 (document processing). This study replaces the model and prompt recommendation in §5.6–5.9 with a faster, cheaper set-up that is at least as accurate.
The bar	At least **98% line accuracy** across all documents, with the 95% interval also at or above 98%, and **zero errors left unflagged**. Only configurations that clear the bar compete on time and cost.
What was run	27 documents (17 used while tuning, 10 held out), each read 3 times per configuration. 24 configurations across Gemini 3.8 Flash, Gemini 3.5 Flash-Lite, Gemini 4 Argon, Claude Haiku 5.5 and Claude Sonnet 5.5, plus routing and escalation variants. 1,251 extraction jobs, 1,555 model calls.
Spend	$9.78 of API usage for this study ($35.03 for all live testing since the audit). The temporary keys were never written to any repository and are deleted from the test machine.

1. Summary

Recommendation: pipeline A. Typed documents (Word, workbooks, PDFs with a text layer) go to Gemini 3.8 Flash in *line-reference* mode: the app numbers every printed line and the model answers with line numbers instead of copying text, using short output keys. Long documents are first split at recipe starts. Photos go to Gemini 3.8 Flash at medium resolution, and Claude Haiku 5.5 reads each photo in parallel as a second opinion. Two new code checks run on every line. On all 27 documents this measured **99.6% line accuracy** (95% interval 99.4%–100.0%), **no unflagged errors**, a median of **5.0 s** per document, **27 s** at the 95th percentile, and **$0.0073 per document**.

	Accuracy	Unflagged errors	Median / 95th pct time	Cost per document	Lines staff must check
M3 as delivered	85.1%	114	1.6 s / 13 s	$0.0036	0% (errors are silent)
Audit brief (evidence-first + 3.8 Flash)	99.5%	0	6.7 s / 58 s	$0.0101	10.6%*
A. Recommended	**99.6%**	**0**	**5.0 s / 27 s**	**$0.0073**	**1.5%**

* With the audit brief's code checks; measured on pipeline A's outputs.

- **A beats the audit brief's set-up on every measure:** 28% cheaper per document, the 95th-percentile wait falls from 58 s to 27 s, and accuracy holds (99.6% vs 99.5%).
- **Staff checking falls about 7×.** With the audit brief's checks, about 1 extracted line in 10 is flagged for a person to look at, most of them correct. Two code checks and the second read on photos bring that to 1.5% of lines (about 1 in 66). Every wrong line was still flagged.
- **API cost is not the constraint.** Pipeline A costs about $52 a month for 100 locations today, and about $100 after Gemini 3.8 Flash doubles its price on January 1, 2027. Staff time spent checking lines is the cost that matters, which is why the review load gets its own section (§6).
- **Two alternatives also clear the bar.** B puts typed documents on Claude Haiku 5.5: 99.6% accuracy at $0.0033 per document (less than half of A), but slower (95th percentile 55 s). It is the switch to make if volume grows after

the 2027 price rise. C puts typed documents on Gemini 3.5 Flash-Lite: the fastest (median 2.3 s) at 99.0%, with a thinner margin over the bar. **Neither held the bar on the new real documents (§11)**, so both are fallbacks to re-test, not equals.

- **Not worth it:** Claude Sonnet 5.5 (it skips event sheets as "not a recipe", 84.5% overall, about 3× the cost); Haiku without thinking (96.9%, below the bar); prompt caching (our fixed prompt is short); Gemini's half-price Flex tier (median wait 131 s against 7 s, worst 16 minutes); escalation cascades (errors are already flagged).
- **Gemini 4 Argon (released October 13) is the most accurate reader we tested, but not worth making the default.** With Gemini 4 reading typed documents it scored 99.9% (held-out set: 100.0%), but it costs about 3× pipeline A per document (and its price doubles on December 1), is about twice as slow, and is no better on handwriting. Used only to re-read the typed recipe documents that pipeline A flags, it reached 100.0% on this set at about 3× A's average cost (§8).

Update, October 15: tested on 58 new real documents (§11). Recipe files from the owner's archive that no prompt or check was tuned on: old Word and Excel files, Compass recipe prints, a photo and five non-recipes. Pipeline A held the bar on accuracy, **99.3%** (95% interval 98.5%–99.9%), but only after fixing five bugs in M3's file readers. With the readers as delivered, accuracy fell to **84.4%**, with 163 wrong lines left unflagged, mostly from .xls files the app cannot read. With the code checks as frozen, 3 wrong lines (one line read three times) were left unflagged. Two new code checks catch them. **B and C did not hold the bar on these documents** (97.7% and 95.7%), so A is the only configuration that cleared it on every set.

2. The bar and how it is measured

Line accuracy = lines extracted exactly right ÷ (lines the document contains + wrong, invented or merged lines produced). A missed line and a wrong line both count against it, so a model cannot score well by returning less. "Exactly right" means ingredient name, amount and unit for recipes, and description, quantity, unit price and line total for invoices.

- **Answer keys.** Hand-made for the short recipes, handwritten cards and invoices. For the four long documents (two cookbook chapters, two event sheets) the key is built from the printed lines by a script and was then corrected by hand. The generated invoices carry their own key, written from the same data that drew them.
- **Unflagged error** = a wrong line that the code checks did not mark amber. The bar allows none. A flagged error costs a cook a few seconds on the review screen. An unflagged error goes silently into food cost.
- **Development and held-out sets.** Prompts and checks were tuned on 17 documents. 10 more were held out: six recipe documents that keep our formats with new content (a second event sheet, a second cookbook chapter, four single recipes including a .docx), and four generated invoices (a clean template with fees, a distributor layout with catch-weight lines, a credit memo, and a photographed produce invoice).
- **Runs and intervals.** Every configuration read every document 3 times; results pool the runs. The 95% interval is a bootstrap over documents (whole documents are resampled, because errors cluster in a few of them), so it reflects how much the result depends on which documents are in the set.
- **Time** is wall-clock per document, including file preparation, from one machine with several configurations running at once. Treat it as relative, not as a production guarantee. **Cost** is computed from the token counts each API returned, at list prices on October 12, 2026, including cached-token and Flex discounts where they applied.

Disclosure: changes made after seeing results. Two code checks (§6) were designed after the second round of runs, and one invoice rule ("row has no amounts") after the first held-out pass. All three are generic rules, not fixes for one document, and none changes accuracy (they only change which lines are flagged). The new real documents in §11 were their first test on documents nobody had looked at, and they held up.

3. Results

Configurations that clear the bar (accuracy and the low end of its interval both at least 98%) are blue or green. Every configuration except M3 flags every error it makes; the differences are in how many lines they get right, how long they take and what they cost. The full table, with development and held-out sets shown separately, is Appendix A.

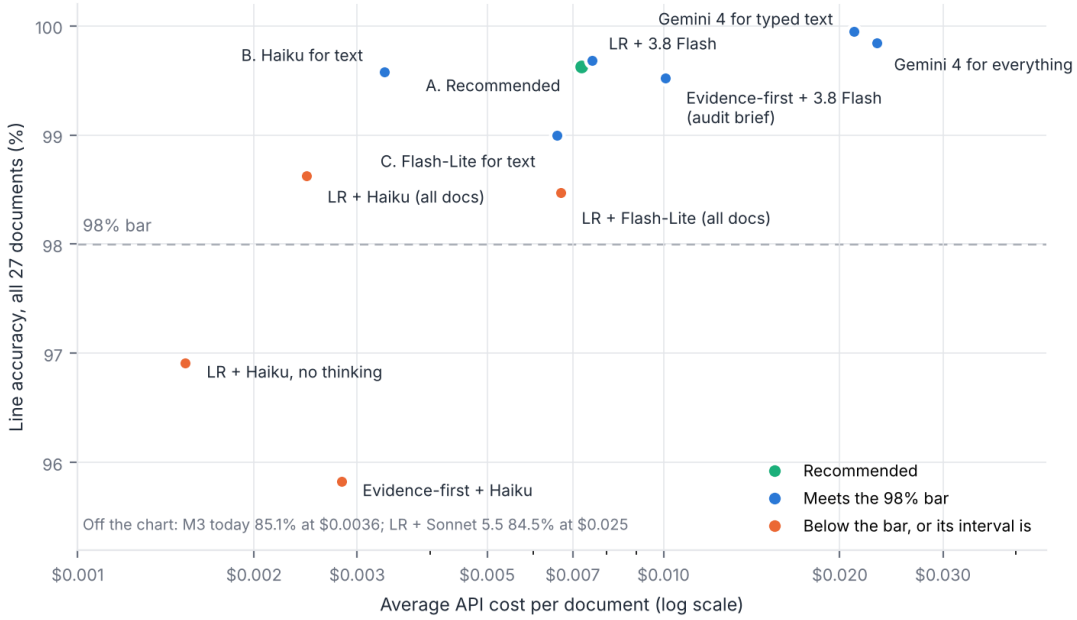


Figure 1. Line accuracy against average API cost per document, all 27 documents, 3 runs each. A, B and C include the Haiku second read on photos.

Configuration	Accuracy (95% interval)	Held-out	Unflagged	Median	95th pct	Cost/doc	Verdict
A. 3.8 Flash, line-reference, compact + split; photos 3.8 medium	99.6% 99.4%–100.0%	99.4%	0	5.0 s	27 s	$0.0073	Recommended
B. Haiku 5.5 for typed text; photos 3.8 medium	99.6% 98.8%–100.0%	99.1%	0	8.1 s	55 s	$0.0033	Cost option
C. Flash-Lite for typed text; photos 3.8 medium	99.0% 98.3%–99.7%	98.6%	0	2.3 s	21 s	$0.0066	Speed option
Gemini 4 Argon for typed text; photos 3.8 medium	99.9% 99.8%–100.0%	100.0%	0	9.3 s	44 s	$0.0212	Meets bar; 3× A's cost, slower
Gemini 4 Argon for everything	99.8% 99.4%–100.0%	100.0%	0	9.3 s	44 s	$0.0231	Meets bar; 3× A's cost, slower
Line-reference + 3.8 Flash, all documents	99.7% 99.4%–100.0%	99.6%	0	6.3 s	46 s	$0.0075	Meets bar
Evidence-first + 3.8 Flash (audit brief)	99.5% 98.9%–100.0%	99.2%	0	6.7 s	58 s	$0.0101	Superseded
Line-reference + Haiku 5.5, all documents	98.6% 95.9%–99.8%	99.1%	0	7.7 s	55 s	$0.0025	Fails: interval reaches 95.9% (handwriting)
Line-reference + Flash-Lite, all documents	98.5% 96.0%–99.6%	98.7%	0	2.4 s	21 s	$0.0067	Fails: interval reaches 96.0% (handwriting)

Configuration	Accuracy (95% interval)	Held-out	Unflagged	Median	95th pct	Cost/doc	Verdict
Line-reference + Haiku 5.5, no thinking	96.9% 94.2%–98.3%	96.9%	0	3.8 s	32 s	$0.0015	Below bar
Evidence-first + Haiku 5.5	95.8% 90.9%–99.2%	95.0%	0	7.9 s	64 s	$0.0028	Below bar
Line-reference + Sonnet 5.5	84.5% 60.5%–99.7%	88.5%	0	4.9 s	61 s	$0.0246	Below bar (skips event sheets)
M3 as delivered	85.1% 66.5%–92.5%	85.1%	114	1.6 s	13 s	$0.0036	Below bar; errors silent

Table 1. All 27 documents, 81 jobs per row; "Held-out" = the 10 held-out documents only; code checks v2 throughout. Rows that read photos with a second model include the Haiku read (slower read's time, both reads' cost).

4. What moved the needle

Each lever was measured on its own against the same base (line-reference + 3.8 Flash, all documents) so its effect can be read directly. Percentages are relative changes.

Lever	What changes	Accuracy	Time	Cost	Verdict
Line-reference instead of copied evidence	Model returns line numbers; code fills in the printed line	99.5% → 99.7%	95th pct 58 → 46 s	-25%	Adopt
Compact output keys	One-letter JSON keys, expanded in code; output tokens −19%	99.7% → 99.6%	median 6.3 → 5.3 s	-12%	Adopt
Split long text at recipe starts	A small first call lists where recipes begin; pieces run in parallel	99.7% → 99.7%	95th pct 46 → 30 s	+8%	Adopt
Compact + split together		99.5%	5.0 s / 28 s	-6%	Adopt
Photos at medium resolution	Gemini media resolution MEDIUM instead of HIGH; photo input tokens −42%	99.7% → 99.8%	no change (re-checked unloaded)	−13% on photos	Adopt
Code checks v2	Line accounting + unit-inside-name (§6)	no change	no change	none	Adopt: review 10.6% → 7.3%
Second read of photos by Haiku 5.5	Photo lines turn green only when two vendors agree	no change	none (runs in parallel)	+$0.0017 per photo	Adopt: review 7.3% → 1.5%
Thinking level	3.8 Flash has no "minimal"; Flash-Lite low vs minimal: 98.4% vs 98.5%	–	–	–	Not a lever
Haiku 5.5 with thinking off		98.6% → 96.9%	median 7.7 → 3.8 s	-38%	Reject: below bar
Prompt caching	Claude: 80% cache hits in a busy queue. Gemini: fixed prompt too short to cache	none	none	−6 to −11% on Claude	Enable on Claude only
Gemini Flex tier	Same model, best-effort queue at half price (16 matched jobs)	same model	median 131 s vs 7 s; worst 952 s	−50%	Bulk back-catalog only

Lever	What changes	Accuracy	Time	Cost	Verdict
Batch APIs (Gemini, Claude)	Results within 24 hours	same model	up to 24 h	−50%	Bulk back-catalog only
Escalation cascade	Cheap model first; 3.8 Flash re-reads when anything is flagged (Haiku → 3.8: 18 of 66 documents escalated)	99.7%	95th pct 57 s	$0.0050	Not needed: errors are already flagged

Table 2. Lever effects on all 27 documents. Base for rows 1–5: line-reference + 3.8 Flash (99.7%, 6.3 s median, 46 s 95th pct, $0.0075 per document). Flex compared on the same documents and runs.

Why line-reference wins

In the audit brief's evidence-first prompt, the model copies each printed line as proof and code checks the copy against the document. That proof is what makes errors visible, but copying costs output tokens, and output tokens are most of both the cost and the time. In line-reference mode the app numbers the lines it already extracted ("[12] 2 cups sugar") and the model answers with `lines: [12]`. Code fills in the printed line, so the proof is exact by construction: the model can no longer paraphrase its evidence. Photos and scanned PDFs have no text to number, so they keep the evidence-first prompt.

Why split at recipe starts, not at a fixed size

Splitting a long document into fixed-size pieces made the 47-second chapter about 3× faster, but it cut recipes in half: 21 of 139 lines landed in the wrong recipe. Pipeline A first asks Flash-Lite (minimal thinking, under a second, about $0.002) for the line where each recipe begins, then splits only there. Across every split run (18 document-runs across three configurations) **no line moved to a different recipe** compared with the unsplit read. The slowest held-out chapter dropped from about 50 s to about 11–16 s.

5. Where the time goes

Almost all of the wait is the model writing its answer. Gemini 3.8 Flash wrote about 127 output tokens per second in these runs, and Flash-Lite about 414. A single recipe needs 400–900 output tokens; a cookbook chapter or a dense event sheet needs 4,500–8,000, which is 30–50 seconds on 3.8 Flash in one call. So the levers that work are fewer output tokens (line-reference, compact keys) and parallel pieces (split at recipe starts). Thinking settings made no difference here.

Document type	A (3.8 Flash)	B (Haiku text)	C (Flash-Lite text)
Short recipe (Word/PDF text)	4.1 s	7.6 s	1.9 s
Long multi-recipe document	23.2 s	53.2 s	7.5 s
Workbook (recipe sheets)	11.3 s	18.8 s	6.4 s
Handwritten recipe photo	about 25 s (Gemini 12–15 s; Haiku 20–25 s)	same as A	same as A
Invoice PDF (text layer)	4.6 s	6.2 s	1.8 s
Invoice photo	about 5–6 s (both reads)	same as A	same as A

Table 3. Median seconds per document. Photo times: the Gemini read from a separate re-check run one document at a time; the Haiku second read from the main runs. A photo is ready when the slower read finishes, so Haiku sets the pace on handwritten cards. One handwritten card once took 96 s in a single call: rare tail delays happen, so queue time-outs should be at least 120 s, with one retry.

Pipeline A's slowest documents are dense event sheets (about 22–35 s). Each one is a single "recipe" of 40 short dishes, below the split threshold. Lowering the split threshold for documents with many short sections is the next time lever. It is not yet measured.

6. Cutting the lines staff must check

A flagged line costs a person a few seconds; a flagged line that was right all along is pure waste. With the audit brief's checks, about 1 line in 10 was flagged, mostly lines that were correct. Almost all of those came from two places, and both are fixable in code:

- **Shared lines on event sheets.** "Vanilla Morel Demi: ½ lb Dried Morels, 2 Bottles Champagne, … Veal Demi, Rosemary, Mirepoix" holds several ingredients, some without an amount. The old check flagged "Rosemary" because its line shows amounts, even though those amounts belong to its neighbours. **Line accounting**: flag an entry with no amount only when its printed line shows more amounts than the entries sharing that line have claimed.
- **Photos.** A photo has no text layer, so nothing can be checked against the page, and every photo line was flagged. **Second read**: Claude Haiku 5.5 reads the same photo in parallel. A line turns green only when both vendors give the same name, amount and unit (and price, on invoices). Two different vendors rarely misread the same handwriting the same way. In these runs, no line both models agreed on was wrong.
- **Unit inside the name** (new check). Rarely, a model puts the unit into the name ("slices smoked salmon", no unit; "ears of Corn"). That costs a recipe wrongly, and the old checks let it through. It happened in one of the Flex-tier runs. The check flags a line whose unit is empty and whose name starts with a unit word.

Pipeline	Audit-brief checks	+ line accounting, unit-in-name	+ second read on photos	Wrong lines left green
A. Recommended	10.6%	7.3%	**1.5%**	0
B. Haiku for text	10.6%	7.2%	**1.1%**	0
C. Flash-Lite for text	10.6%	7.8%	**1.8%**	0

Table 4. Share of all extracted lines shown amber on the review screen (all 27 documents, 3 runs). For comparison, M3 as delivered flags nothing and leaves 114 wrong lines green.

7. What it costs

Cost per document depends mostly on how much the model has to write, so it is shown by document type. Monthly figures use assumed volumes; change them to your own. "2027" applies Gemini 3.8 Flash's announced January 1, 2027 price ($1.50 / $7.50 per million input / output tokens, double today's).

Document type	A	B	C	Audit brief
Short recipe (Word/PDF text)	$0.0026 2027: $0.0053	$0.0011 2027: $0.0011	$0.0023 2027: $0.0023	$0.0041 2027: $0.0081
Long multi-recipe document	$0.0255 2027: $0.0504	$0.0085 2027: $0.0085	$0.0230 2027: $0.0230	$0.0373 2027: $0.0745
Workbook (recipe sheets)	$0.0189 2027: $0.0378	$0.0068 2027: $0.0068	$0.0167 2027: $0.0167	$0.0317 2027: $0.0634
Handwritten recipe photo	$0.0080 2027: $0.0138	$0.0080 2027: $0.0138	$0.0080 2027: $0.0138	$0.0070 2027: $0.0140
Invoice PDF (text layer)	$0.0028 2027: $0.0057	$0.0010 2027: $0.0010	$0.0025 2027: $0.0025	$0.0041 2027: $0.0082
Invoice photo	$0.0041 2027: $0.0072	$0.0041 2027: $0.0072	$0.0041 2027: $0.0072	$0.0033 2027: $0.0066

Table 5. Average API cost per document. A, B and C include the Haiku second read on photos.

Scenario (documents)	A	B	C	Audit brief	M3 today
Onboard one restaurant (one-off) 120 short recipe, 10 long multi-recipe document, 3 workbook, 25 handwritten recipe photo	$0.83 2027: $1.60	$0.44 2027: $0.59	$0.76 2027: $0.90	$1.13 2027: $2.26	$0.37 2027: $0.37
One location, a normal month 10 short recipe, 1 long multi-recipe document, 3 handwritten recipe photo, 90 invoice pdf, 30 invoice photo	$0.45 2027: $0.87	$0.26 2027: $0.37	$0.42 2027: $0.53	$0.57 2027: $1.13	$0.18 2027: $0.18
100 locations, a normal month (+8 onboardings) 1960 short recipe, 180 long multi-recipe document, 24 workbook, 500 handwritten recipe photo, 9000 invoice pdf, 3000 invoice photo	$51.97 2027: $99.81	$29.11 2027: $41.33	$48.06 2027: $60.28	$65.53 2027: $131.06	$21.05 2027: $21.05

Table 6. API cost per scenario. M3 is cheapest only because it does not check its own work.

Reading the numbers. Even at 100 locations the whole difference between the options is tens of dollars a month. One cook-hour checking flagged lines costs more than a month of extraction for a single location. That is why the recommendation favours accuracy and a low review load over the cheapest model. Revisit B after January 1 if volume is in the thousands of locations. Gemini 4 Argon does not change the picture (§8).

8. Models, including Gemini 4

Model	Price per 1M tokens (in / out)	What we measured	Role
Gemini 3.8 Flash	$0.75 / $3.75 (from Jan 1, 2027: $1.50 / $7.50)	Most accurate on everything, best on handwriting. Writes about 127 tokens/s.	Default for all routes
Gemini 3.5 Flash-Lite	$0.30 / $2.50	Fastest (about 414 tokens/s). 99.0% on typed text (C), but weaker on handwriting (lasagna card 50 of 57 lines) and a distributor invoice.	Speed option for typed text; outline call
Claude Haiku 5.5	$0.10 / $0.50	Cheapest. 99.6% on typed text (B), but slow with thinking on, and below the bar with it off. Misreads handwriting (pavlova card 27 of 33).	Second reader on photos; cost option for text
Claude Sonnet 5.5	$2 / $10	Strong on single recipes, but labels event sheets "other" or "menu" and returns little or nothing from them (84.5% overall). About 3× A's cost.	Not recommended
Claude Opus 5.5	$4 / $20	Not tested: twice Sonnet's price, and Sonnet showed no accuracy gain to buy.	Not recommended
Gemini 4 Argon	$2 / $10 (from Dec 1, 2026: $4 / $20)	Released Oct 13, tested the same day. Most accurate on typed text (100.0% on the held-out set), no better than 3.8 Flash on handwriting, writes about 74 tokens/s (about half 3.8 Flash's speed).	Not the default; optional re-reader for flagged recipe documents

Table 7. List prices on October 13, 2026.

Pin model versions. M3 calls `gemini-flash-lite-latest`, an alias that Google repoints without notice, so extraction quality can change overnight. Use explicit model IDs and change them only after the harness passes.

Gemini 4 Argon, tested on release day

Google released Gemini 4 Argon on October 13. It ran through the same 27 documents, 3 times each, with pipeline A's settings (line-reference with compact keys and splitting for typed text, evidence-first at medium resolution for photos). Four ways to use it, against pipeline A as deployed:

Set-up	Accuracy	Held-out	Unflagged	Median / 95th pct	Cost/doc	100 locations/month (now / 2027)
A. Recommended (3.8 Flash)	99.6%	99.4%	0	5.0 s / 27 s	$0.0073	$52 / $100
Gemini 4 for typed text; photos 3.8 Flash	99.9%	100.0%	0	9.3 s / 44 s	$0.0212	–
Gemini 4 for everything	99.8%	100.0%	0	9.3 s / 44 s	$0.0231	$177 / $349
3.8 Flash for typed text; photos Gemini 4	99.5%	99.4%	0	5.1 s / 31 s	$0.0092	–
A, then Gemini 4 re-reads flagged typed recipe documents (17 of 66 typed document-runs)	100.0%	100.0%	0	5.2 s / 67 s	$0.0208	$107 / $210

Table 8. Gemini 4 Argon against pipeline A, all 27 documents, 3 runs, code checks v2. "2027" prices both Gemini 4's December 1 and 3.8 Flash's January 1 increases. Cost per document is at today's prices; every row includes the Haiku second read on photos.

Decision rule applied. (1) Accuracy and its interval clear 98%: yes. (2) No unflagged errors: yes. (3) Handwriting no worse than 3.8 Flash: about the same (one more wrong line on the lasagna card in 9 photo reads), so photos stay on 3.8 Flash. (4) Cheaper or faster at equal accuracy: no, it costs about 3× per document and its median wait is nearly twice pipeline A's, because it writes about 74 tokens a second against 3.8 Flash's 127. **Verdict: pipeline A stays the recommendation.**

Where Gemini 4 earned its keep. Pipeline A's remaining misses were 5 lines across the two cookbook chapters and 2 flagged wrong lines on an event sheet. Gemini 4 got all of them right. Having it re-read only the typed recipe documents that pipeline A flags fixed every one on this set, at about 3× A's average cost and a longer wait for those documents. That is worth considering later, for example as a "read again with the stronger model" button on the review screen, if missed lines on long documents turn out to matter in practice. It is not needed to clear the bar.

Testing the next model

The same command tests any future model. From `backend/`, with the API key in the environment and `<id>` the model ID the vendor publishes:

```
python3 ../audit/extraction-benchmark/gold.py      # writes the case lists (once per checkout)
EF_SHEETS=1 EF_IMAGES=auto EF_MEDIA=MEDIUM LR_COMPACT=1 LR_OUTLINE=1 LR_CHUNK=4000 \
  npx tsx ../audit/extraction-benchmark/runner.ts "lr-gemini:<id>" \
  ../audit/extraction-benchmark/cases_all.json ../audit/extraction-benchmark/out/study/<name> 3 3
# then add its folder to CONFIGS in study.py and its price to PRICE in score.py, and run:
cd ../audit/extraction-benchmark && CHECKS_V2=1 python3 study.py
```

Decision rule. Adopt a new model for a document type only if, on both the development and held-out sets: (1) accuracy is at least 98% and its 95% interval stays at or above 98%; (2) no error is left unflagged; (3) handwriting is no worse than 3.8 Flash's; and (4) at equal or better accuracy it is cheaper or faster than pipeline A, priced against 3.8 Flash's 2027 price. About 81 jobs cost a few dollars and take under an hour.

9. The recommended pipeline, for the developers

Step	What happens	Model / setting
1. Prepare	Read the file in code. Word, workbooks and PDFs with a usable text layer become numbered lines ("[n] text", blank lines skipped). Images, PDFs that carry photos, and PDFs with under 80 characters of text go to the photo route.	No model
2a. Typed text	One line-reference call with compact keys. If the text is over about 5,200 characters: an outline call first, then split at recipe starts into about 4,000-character pieces, read in parallel. Line numbers stay global, so the pieces simply concatenate.	Gemini 3.8 Flash, thinking low; outline on 3.5 Flash-Lite, thinking minimal
2b. Photos	Evidence-first call at medium resolution, plus the same prompt to a second vendor in parallel. Lines both agree on (name, amount, unit, and price on invoices) are green.	Gemini 3.8 Flash, mediaResolution MEDIUM; Claude Haiku 5.5 with cached system prompt
3. Check	Code checks on every line, including line accounting and unit-inside-name; missed-line check on typed text (a printed line that starts with an amount but is covered by no entry).	No model
4. Review	Amber lines first, with the printed line beside each. Invoices: computed prices and totals that do not add up are amber.	No model

Implementation notes

- **Fix the file readers first (§11).** On new real documents these five reader bugs cost more accuracy than any model choice: (1) .xls workbooks are not read at all (add the format, or convert to .xlsx); (2) Word's automatic list numbers are dropped, so "1 cup" becomes "cup" (read `numbering.xml`); (3) manual line breaks (`<w:br/>`) are ignored; (4) a Word file that holds only an image is sent as empty text (send the image to the photo route); (5) a PDF with a good text layer and a logo goes to the photo route (route by text layer, not image count). The harness's `READER_FIX` code in `runner.ts` is a working reference for each.
- **Amounts real documents print.** Sum compound amounts ("1 lb, 2 oz", "1/3 cup, 2 tbsp, 1-1/4 tsp", "1 T + 2 tsp") into one base unit; read the fraction slash ("1⁄4" is 1 1/4); drop a leading "<"; accept "Q" for quart; and move a leading counting word ("cloves", "slices", "sprigs") from the name to the unit when no unit was read.
- **Where.** `backend/src/modules/documents/extraction.ts`: add a line-reference extractor beside `GeminiExtractor`. `prepareInput` and `textChunks` already produce the text. Numbering, resolving line numbers back to printed lines, compact-key expansion and the outline split come to about 120 lines; the harness's `runner.ts` (`numbered`, `resolveLines`, `expandCompact`, `outlineStarts`, `splitAtStarts`) is a working reference.
- **Prompts.** The line-reference prompt is the evidence-first v3 prompt (audit report, Appendix B) with rule 2 replaced. The replacement rule, the compact-key note and the outline prompt are in Appendix B here. The schema swaps `evidence` for `lines: integer[]`.
- **Number words.** Parse "one", "two", "half", "a dozen" as amounts. Recipes written as prose ("Season one pound of Raw Tuna…") otherwise produce unparsed amounts.
- **Usage logging.** Store each call's token counts, cached tokens and service tier on the document job. That is what made this study's cost figures exact, and it is how a price change or a runaway document shows up in production.
- **Caching.** Put Claude's system prompt in a `cache_control` block (5-minute cache, 80% hit rate in a busy queue). On Gemini there is nothing to do: the fixed prompt is below the size where caching pays.
- **Time-outs.** Allow 120 s per call with one retry; one photo call took 96 s in an otherwise 15-second series.
- **Show the photo read early.** Haiku's second read is slower than Gemini's on handwriting. The review screen can show Gemini's lines as soon as they arrive, all amber, and turn agreed lines green when Haiku finishes.
- **Regression gate.** Run the harness (Appendix C) before any prompt, model or check change ships. The bar is the one in this report.

Code checks v2 (sketch)

```
// 1) Line accounting: an entry with no amount on a shared printed line is only suspect when the
//    line shows more amounts than the entries sharing it have claimed.
const shown = (printedLine.match(AMOUNT_WITH_UNIT) ?? []).length
const claimed = entries.filter((e) => sameLines(e, entry) && e.quantityText).length
if (!entry.quantityText && !entry.amountNote && shown > claimed)
  amber('evidence shows an amount the line lacks')

// 2) Unit inside the name: "slices smoked salmon" with no unit, "ears of Corn".
const first = entry.nameText.trim().split(/\s+/)[0].toLowerCase().replace(/[(,.]/g, '')
if (!entry.unitText && UNIT_WORDS.has(first)) amber('unit inside name')
// UNIT_WORDS: slice(s) bunch(es) ear(s) cup(s) clove(s) pinch head(s) stick(s) bag(s)
//   sheet(s) can(s) oz ounce(s) lb(s) pound(s) sprig(s) dash piece(s) scoop(s) bottle(s)
//   tbsp tsp quart(s) pint(s) gallon(s) g kg ml liter(s) pack(s) packet(s) box(es) case(s)
//   jar(s) container(s) clamshell(s) flat(s)

// 3) Photos: green only if a second vendor's read has the same name (similarity >= 0.5), amount and unit
//    (count units interchangeable), and on invoices the same price.
```

10. Risks and what to do next

- **Test sets.** The 58 real archive documents (§11) are now the main regression set beside the original 27. Still missing: real distributor invoices and phone photos taken in a kitchen. Collect and key those next.
- **Checks tuned after seeing results.** Line accounting and unit-inside-name held up on the new documents. The two checks and the counting-word fix added in §11 were written after seeing its errors, so the next batch of documents is their real test.
- **Models change under us.** Providers update models and aliases. Pin IDs, log usage, and re-run the harness monthly and on every model change.
- **Price changes.** Gemini 3.8 Flash doubles on January 1, 2027. Pipeline A's monthly cost doubles with it; it stays small, but re-check B (Haiku for typed text) then. Gemini 4 Argon's own price doubles on December 1.
- **Stricter models and the wrong-document rule.** Sonnet 5.5 refused event sheets under the rule that tells the model to stop on non-recipe documents. If the team moves to a stronger Claude model, that rule's wording needs re-testing on event sheets first.
- **Measured time is relative.** Latency came from one machine with many runs at once. Production times will differ; the ranking between options should not.

11. New real documents (October 15)

The owner shared recipe folders from the restaurant archive. We chose 58 files that no prompt, check or setting had been tuned on and wrote the answer key for each from the document itself before any model ran: 38 Word files (old .doc and newer .docx), 9 Excel workbooks (most in the old .xls format), 5 Compass "Recipe Print" PDFs with sub-recipes and compound amounts, a Word file that holds only a flow-chart image, and 5 files that are not recipes (two menus, a vendor product sheet, a blank template, a plated-dish photo), which should produce no lines. The key has **775 lines**. A 66,000-character banquet production packet was scored separately against the 840 amounts printed in it. Each configuration read every document 3 times (912 jobs, $4.07 of API usage).

Real documents print amounts the earlier set did not: compound amounts ("2 lb, 4 oz", "1/3 cup, 2 tbsp, 1-1/4 tsp"), Word's built-up fractions (1¼ stored as "1" then "1⁄4", read as "11⁄4"), "<1/8 tsp", "2 Q", units in a column header ("gr."), and tables whose names and amounts sit in separate cells. The scorer was extended for these and re-checked: every number reported for the original 27 documents is unchanged. After the first runs every disagreement between a model and the key was checked against the document. One key line was wrong (a document's typo, "earscorn", that models copy faithfully) and was corrected; every other disagreement was a model error and stands.

Configuration	Accuracy (95% interval)	Wrong lines	Unflagged	Median / 95th pct	Cost per doc	Long packet: amounts found
A, readers as delivered	84.4% (71.3%–93.7%)	195	163	3.0 s / 9 s	$0.0059	97%
A, readers fixed	99.3% (98.5%–99.9%)	9	3	2.8 s / 8 s	$0.0034	98%
B (Haiku for typed text)	97.7% (95.0%–99.5%)	10	9	5.6 s / 17 s	$0.0013	3%
C (Flash-Lite for typed text)	95.7% (92.3%–98.1%)	45	16	2.0 s / 6 s	$0.0030	92%
gpt-6-luna for typed text	97.4% (94.7%–99.2%)	24	10	6.8 s / 21 s	$0.0006	8%

Table 9. 58 new documents, 3 runs each, code checks as frozen before this set. Photos (2 documents) on 3.8 Flash at medium resolution in every row; the Haiku second read on them adds under $0.0001 per document on average. Every configuration returned nothing for all 5 non-recipes, every time. The long packet is not in the accuracy columns; on it, A and C made no wrong lines.

- **Pipeline A cleared the accuracy bar, with a margin:** 99.3%, interval 98.5%–99.9%, median 2.8 s, $0.0034 per document. On the long packet it found 98.1% of the printed amounts with no wrong line (14 s, about $0.17 for that one file).
- **Its 3 unflagged errors were one printed line, "Juice of 1 orange"**, read without the amount once and with "juice of" as the unit twice. The other 6 wrong lines were all flagged.
- **B, C and gpt-6-luna fell below the bar.** Haiku misaligned a table whose names and amounts sit in separate cells and twice stopped after the first page of a four-page PDF. Flash-Lite dropped column-header units and kept "salt, pepper and sugar" as one entry. gpt-6-luna took bottle sizes ("Port 1.5 Liter") for amounts and dropped the second part of compound amounts. Each also left 9–16 wrong lines unflagged.
- **Haiku and gpt-6-luna gave up on the long packet** (3% and 8% of amounts) because they read it in one call and stop early. Only the Gemini path splits long text at recipe starts; they would need the same split.

The file readers mattered more than the model

Documents	Files	A, readers as delivered	A, readers fixed
Word	38	94.0% (40 unflagged)	99.0% (3 unflagged)
Excel workbooks	9	62.0% (123 unflagged)	100.0% (0 unflagged)
Compass PDFs	5	100.0% (0 unflagged)	100.0% (0 unflagged)

Table 10. Same model and prompt; only the code that turns the file into text changed.

M3 does not read .xls at all: the workbook's raw bytes go to the model as text, and the model returns confident, wrong amounts that the checks cannot catch, because the garbage it quotes is really in that "text". In 3 of the 18 .docx recipes, Word's automatic list numbering held the amounts, and the reader drops it, so "1 teaspoon cayenne" arrives as "teaspoon cayenne". The Compass PDFs read correctly either way: the photo route they fall into today is about as accurate as the text route, just slower and dearer (2.6 s and $0.0037 on average with the text layer).

Two checks and a code fix added after this set

Pipeline A with	Accuracy (95% interval)	Wrong lines	Unflagged	Lines to check
Checks as frozen (v2)	99.3% (98.5%–99.9%)	9	3	2%
+ unit the app cannot cost; number printed but no amount read	99.3% (98.5%–99.9%)	9	0	4%
+ counting word moved from name to unit (code)	99.7% (98.9%–100.0%)	3	0	3%

Table 11. New documents, readers fixed. The share of lines to check includes the flagged correct lines.

Unit the app cannot cost turns a line amber when its unit is not in the app's own unit table, since such a line cannot be costed without a conversion anyway (the table is in `backend/src/costing/units.ts`). **Number printed but no amount read** catches "Juice of 1 orange". Together they leave no error unflagged on this set. On the original 27 documents they change no accuracy figure but raise the share of lines to check from 7% to 10%, mostly event-sheet units such as clamshells, flats and packets. The **counting-word fix** is plain code: when no unit was read and the name starts with one ("cloves garlic", "slices of tomato"), move it to the unit. It removed 6 flagged errors. All three were written after seeing this set's errors, so the next batch of documents is their real test.

Tried and not adopted

- **A repair call on flagged lines.** A small 3.8 Flash call re-read each flagged line. It fixed the counting-word lines, but it sees only the one line, so it removed units printed in a column header and gave a column table's amount to the wrong ingredient. Those repaired lines then passed every check: 13 wrong lines, all 13 unflagged (98.8%). The code fix above does the useful part without the risk.
- **Size routing.** Sending documents under 1,000 characters to Flash-Lite kept the bar (99.4%, no unflagged errors) but saved 0.6 s at the median and almost nothing in cost. Not worth a second code path.
- **A second vendor on the Compass PDFs** (read as photos, as M3 would today): Haiku agreed with 3.8 Flash on all 129 lines and both were fully right, so the text route needs no second read.

What changes. Pipeline A stays the recommendation, with the reader fixes as the first item to build (§9). B and C are no longer equal alternatives: re-test them on this set before switching to either. The new set joins the regression gate.

Appendix A. Full results

Configuration	All	95% interval	Dev	Held-out	Wrong lines	Not flagged	Clean docs	Median s	95th s	Cost/doc
M3 today (gemini mode, default alias)	85.1%	66.5%–92.5%	85.0%	85.1%	114	114	40%	1.6	13	$0.0036
Evidence-first + Gemini 3.8 Flash (brief's recommendation)	99.5%	98.9%–100.0%	99.7%	99.2%	2	0	98%	6.7	58	$0.0101
Evidence-first + Gemini 3.5 Flash-Lite	93.6%	85.8%–98.1%	95.9%	90.2%	41	0	77%	2.7	23	$0.0068
Evidence-first + Claude Haiku 5.5	95.8%	90.9%–99.2%	96.4%	95.0%	5	0	81%	7.9	64	$0.0028
Line-reference + Gemini 3.8 Flash	99.7%	99.4%–100.0%	99.7%	99.6%	1	0	99%	6.3	46	$0.0075
Line-reference + Gemini 3.5 Flash-Lite (low)	98.4%	96.0%–99.4%	98.0%	98.8%	12	0	93%	2.5	22	$0.0063
Line-reference + Gemini 3.5 Flash-Lite (minimal)	98.5%	96.0%–99.6%	98.3%	98.7%	12	0	94%	2.4	21	$0.0067
Line-reference + Claude Haiku 5.5	98.6%	95.9%–99.8%	98.3%	99.1%	9	0	86%	7.7	55	$0.0025
Line-reference + Claude Haiku 5.5 (no thinking, cached prompt)	96.9%	94.2%–98.3%	96.9%	96.9%	22	0	79%	3.8	32	$0.0015
Line-reference + Claude Sonnet 5.5 (cached prompt)	84.5%	60.5%–99.7%	81.6%	88.5%	6	0	86%	4.9	61	$0.0246
LR + 3.8 Flash, compact output keys	99.6%	99.1%–99.9%	99.5%	99.7%	2	0	98%	5.3	41	$0.0066
LR + 3.8 Flash, long text split at recipe starts	99.7%	99.4%–100.0%	99.6%	99.7%	1	0	99%	6.3	30	$0.0081
LR + 3.8 Flash, compact + split	99.5%	99.2%–99.9%	99.6%	99.4%	3	0	96%	5.0	28	$0.0071
LR + 3.8 Flash, photos at medium resolution	99.8%	99.6%–100.0%	99.9%	99.6%	0	0	100%	6.7	46	$0.0074
Route: text → Flash-Lite LR (minimal); photos → 3.8 Flash	99.1%	98.1%–99.6%	99.4%	98.7%	6	0	96%	2.5	22	$0.0069
Route: text → Flash-Lite LR compact + split; photos → 3.8 Flash	98.9%	98.2%–99.6%	99.1%	98.6%	2	0	96%	2.3	15	$0.0064
Route: text → Haiku LR; photos → 3.8 Flash	99.5%	98.5%–99.9%	99.7%	99.1%	1	0	93%	7.7	55	$0.0032
Route: text → Haiku LR (no thinking); photos → 3.8 Flash	97.5%	95.6%–98.6%	97.9%	96.9%	16	0	84%	4.0	32	$0.0022
A. 3.8 Flash LR compact + split; photos 3.8 medium	99.6%	99.4%–100.0%	99.8%	99.4%	2	0	98%	5.0	27	$0.0069
B. Haiku LR text; photos 3.8 medium	99.6%	98.8%–100.0%	99.9%	99.1%	0	0	94%	8.1	55	$0.0030
C. Flash-Lite LR compact + split text; photos 3.8 medium	99.0%	98.3%–99.7%	99.3%	98.6%	1	0	98%	2.3	18	$0.0063
G4. Gemini 4 Argon, pipeline A settings	99.8%	99.4%–100.0%	99.7%	100.0%	2	0	98%	9.3	44	$0.0228
Route: text → Gemini 4 Argon; photos → 3.8 Flash medium	99.9%	99.8%–100.0%	99.9%	100.0%	1	0	99%	9.3	44	$0.0209
Route: text → 3.8 Flash (A); photos → Gemini 4 Argon medium	99.5%	99.2%–99.9%	99.6%	99.4%	3	0	96%	5.1	31	$0.0089

Accuracy columns are line accuracy. "Clean docs" = share of document-runs with no wrong line and at least 90% of lines found. Rows starting "Route" or A/B/C combine measured runs by document type (typed text from one run set, photos from another); each part ran in full. Code checks v2 throughout.

Appendix B. Prompt changes

B.1 Rule 2 of the evidence-first v3 recipe prompt, replaced for line-reference mode. The rest of the prompt is unchanged; the closing phrase "and the whole printed line as the evidence of each" becomes "and the same line number". The invoice prompt's evidence field becomes "lines: the number(s) of the printed line(s) the row comes from (never copy the row)".

```
2. LINE NUMBERS. The document arrives with a number in square brackets before each printed line
   ("[12] 2 cups sugar"). For every entry give in "lines" the number of the printed line it comes
   from; if its name and its amount are printed on different lines (a table read column by
   column), give both numbers. Never copy the line itself.
```

B.2 Compact output keys (appended to the system prompt; code maps keys back).

```
OUTPUT KEYS. To keep the answer short, the JSON uses one-letter keys: k = documentKind, r = recipes;
in a recipe t = titleText, y = yieldText, p = portionsText, c = components, i = its ingredient entries;
in a component n = nameText, y = yieldText, w = usedAsText; in an ingredient entry l = lines, s = section,
n = nameText, q = quantityText, u = unitText, a = amountNote, g = legibility ("c" clear, "u" unclear).
```

B.3 Outline prompt (Flash-Lite, minimal thinking, schema `{starts: integer[]}`).

```
The document arrives with a number in square brackets before each printed line. Give the number
of the line where each recipe begins: the dish's title line (or its first line when it has no title). A step,
part or component inside a recipe ("Step 2", "For the sauce") is not a new recipe. Give the numbers in order.
```

Appendix C. The harness

Everything in this report can be re-run from `audit/extraction-benchmark/` in the clone repository: the documents, answer keys, the runner (all modes and settings above), the scorer, and the study, cascade, cost-model and review scripts. Raw model outputs are archived in `results/`. API keys are read from the environment and are never stored.

runner.ts	Runs one configuration over a case list. Arms: m3-gemini, m3-claude, ef-gemini, ef-claude, lr-gemini, lr-claude. Settings via environment: EF_MEDIA, LR_COMPACT, LR_OUTLINE, LR_CHUNK, LR_THINK, GEMINI_TIER, CL_CACHE, CL_THINK.
score.py	Scores runs against the keys; code checks (CHECKS_V2=1 for v2); cost from recorded usage.
study.py / cascade.py	Tables 1, 2 and Appendix A; escalation simulations.
costmodel.py / review.py	Tables 5–6 (cost by type and scenario) and Table 4 (review load).
group.py	Checks that splitting never moves a line to a different recipe.
gold.py, autoref.py	Answer keys (development and held-out sets).
gold_new.py, new_study.py	§11: keys for the new real documents (with the audit log of key changes) and its tables. CHECKS_V3=1 and NORMALIZE=1 add the checks and fix from Table 11; READER_FIX=1 in runner.ts turns on the reader fixes; repair.py and size_route_new.py are the two experiments not adopted.